

EdgeOS

A Resource-Aware Operating System for Memory-Constrained Edge Devices

TECHNICAL PROJECT REPORT

STAR-Based Challenge Analysis and Technical Evaluation

Course	OS323 - Operating Systems
Student Name	[Shakib alom fahim]
Student ID	[2222866042]
Faculty	[SSI]
GitHub Repository	[https://github.com/fahimsakib007/EdgeOS]
Video Demonstration	[https://youtu.be/LxQLjPN0wLU]
Submission Date	[29/08/2026]

Executive Summary

EdgeOS is an educational 32-bit x86 kernel prototype created to demonstrate how core Operating Systems mechanisms can cooperate in a memory-constrained edge environment. The final implementation integrates a First-Fit kernel allocator, memory block splitting and coalescing, external-fragmentation measurement, a kernel-side system-call dispatcher, PCB-based process management, Priority and Round Robin scheduling demonstrations, and a resource manager that reacts to low free memory using process priorities.

The strongest measured memory result is a reduction in external fragmentation from 28% to 0% after adjacent free blocks are coalesced. In the resource-pressure experiment, free memory increased from 171,984 bytes to 950,236 bytes after EdgeOS reclaimed the LOW-priority Logger workload. The memory-pressure state changed from CRITICAL to NORMAL while the HIGH-priority Camera and MEDIUM-priority Sensor remained available.

The report also documents the main development challenges using the STAR format requested for submission. These challenges include QEMU output problems under WSL, inefficient early memory allocation, external fragmentation, critical memory pressure, and build/reproducibility issues. The report intentionally distinguishes the implemented prototype from features that are outside its scope, such as ring-3 user processes, interrupt-based user-to-kernel system calls, hardware timer-driven context switching, paging, and virtual memory.

Contents

1. Project Introduction
 2. Problem Statement and Objectives
 3. System Architecture
 4. Technical Implementation
 5. Testing and Evaluation
 6. Challenges Faced - STAR Analysis
 7. Technical Scope and Limitations
 8. Skills Demonstrated
 9. Conclusion
- Appendix A. Final Build and Run Commands

1. Project Introduction

Edge computing devices such as smart cameras, IoT gateways, monitoring nodes and embedded controllers often operate with limited memory and processing resources. When several workloads share a small memory space, poor allocation decisions, external fragmentation and uncontrolled background workloads can reduce the memory available to important tasks.

EdgeOS was developed as a small experimental operating-system kernel to study this problem using real kernel-style components. It boots through GRUB, executes in QEMU, and uses a freestanding C kernel with x86 Assembly boot code. Ubuntu through WSL is used as the development environment, not as the runtime operating system for EdgeOS.

The final project integrates the following components:

- Bootable Multiboot-compatible x86 kernel
- Kernel debug output through QEMU port 0xE9
- 1 MB experimental memory pool
- First-Fit allocation through `kmalloc()`
- Memory release through `kfree()`
- Block splitting and adjacent free-block coalescing
- External-fragmentation measurement
- Kernel-side `SYS_MALLOC` and `SYS_FREE` dispatcher
- PCB-based process management
- Priority and Round Robin scheduling demonstrations
- NORMAL, WARNING and CRITICAL memory-pressure states
- Priority-aware reclamation of a LOW-priority process under CRITICAL pressure

Core project idea

EdgeOS does more than allocate memory. It connects memory state, process priority, scheduling and resource reclamation so the kernel can make a simple resource-aware decision when free memory becomes critically low.

2. Problem Statement and Objectives

2.1 Problem Statement

A memory allocator can answer whether a block is available, but it does not automatically understand workload importance. In a constrained edge device, a background Logger should not necessarily consume memory needed by a Camera or Sensor. Repeated allocation and deallocation can also divide free memory into several smaller regions, producing external fragmentation.

Process	Demonstration Role	Priority
Camera	Critical monitoring workload	HIGH (3)
Sensor	Data collection workload	MEDIUM (2)
Logger	Background logging workload	LOW (1)

The project therefore addresses the following technical question: How can a small experimental kernel manage limited and fragmented memory while using process priority to recover from critical resource pressure?

2.2 Objectives

1. Develop a reproducible bootable x86 kernel.

2. Implement kernel memory allocation and deallocation.
3. Reduce wasted memory using block splitting.
4. Reduce external fragmentation using coalescing.
5. Measure fragmentation quantitatively.
6. Expose memory services through a kernel-side system-call dispatcher.
7. Manage processes through PCBs, states and priorities.
8. Demonstrate Priority and Round Robin scheduling policies.
9. Detect memory-pressure conditions from free memory.
10. Reclaim a LOW-priority process under CRITICAL memory pressure.
11. Produce measurable before-and-after results suitable for a faculty demonstration.

3. System Architecture

The architecture is modular so that each subsystem has a clear responsibility while still interacting with the others. The memory manager owns the experimental pool, the process manager records per-process information, the scheduler makes process-selection decisions, and the resource manager reads memory state and process priority before reclaiming a lower-priority workload.

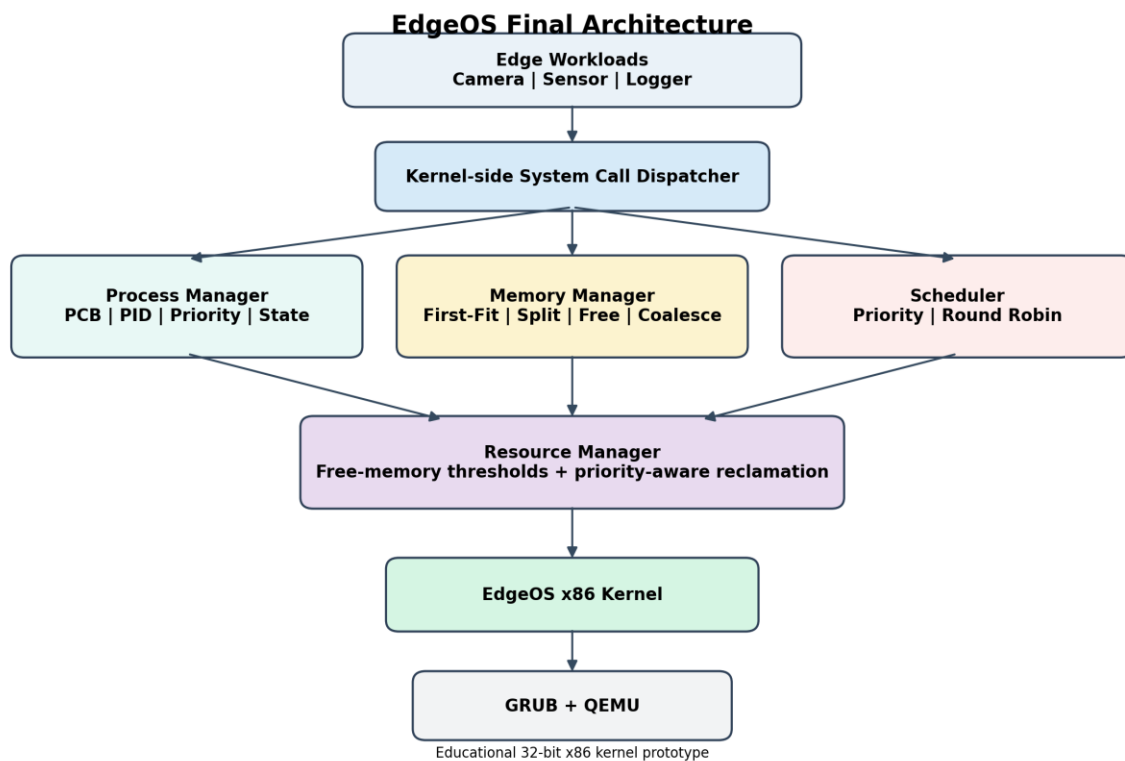


Figure 1. EdgeOS final system architecture.

Module	Primary responsibility	Main interfaces
Boot/Kernel	Start the Multiboot kernel and run the integrated demo	boot.asm, kernel_main()
Debug	Send kernel messages to the QEMU debug console	debug_write(), debug_writeln()
Memory Manager	Allocate, free, split, coalesce and report memory	memory_init(), kmalloc(), kfree(), memory_print()
System Call Dispatcher	Route SYS_MALLOC and SYS_FREE requests	syscall_dispatch()
Process Manager	Maintain PCBs, PID, state, priority and process memory	process_create(), process_terminate()
Scheduler	Demonstrate priority order and cyclic time slices	scheduler_run_priority(), scheduler_run_round_robin()
Resource Manager	Classify memory pressure and reclaim a low-priority process	resource_get_memory_pressure(), resource_handle_memory_pressure()

4. Technical Implementation

4.1 Boot and Debug Path

GRUB loads the Multiboot-compatible kernel image and transfers control through the Assembly boot entry to `kernel_main()`. Because graphical QEMU output was unreliable under WSL during development, EdgeOS uses QEMU debug port 0xE9 and terminal output. This made every subsystem observable during testing.

```
fahim@FAHIM:~/EdgeOS$ qemu-system-i386 \
-cdrom EdgeOS.iso \
-display none \
-debugcon stdio \
-global isa-debugcon.iobase=0xe9

=====
      EdgeOS v1.0
Resource-Aware Edge Operating OS
=====

[BOOT] Kernel initialized successfully.
[BOOT] Debug console initialized.

[MEM] Starting memory manager...
[MEM] Memory manager initialized.
[TEST] Allocating 100 bytes...
[MEM] Memory allocated.
[TEST] Allocation successful.
[TEST] Freeing allocated memory...
[MEM] Memory released.
[MEM] Memory statistics calculated.
[MEM] No memory currently allocated.
[MEM] Free memory available.

[SYSTEM] EdgeOS memory test completed.
```

Figure 2. Early EdgeOS boot and memory test displayed through the QEMU debug console in the terminal.

4.2 Memory Manager

The memory subsystem uses a 1 MB static experimental pool. Each memory block stores metadata including the payload size, free/used status and a pointer to the next block. First-Fit scans from the beginning of the linked list and selects the first free block large enough to satisfy a request.

Design decision - First Fit

First Fit was selected because it is simple to implement, inexpensive to search in a small educational pool, and easy to demonstrate. The project is designed to illustrate OS mechanisms rather than to claim the globally optimal allocator.

When the selected free block is substantially larger than the requested size, the block is split. The requested portion becomes USED and the remainder remains FREE for future allocations. This prevents a small request from consuming an entire large free region.

```

[MEM] Memory manager initialized.

[TEST] Allocating Block A: 100 bytes
[MEM] Free block split.
[MEM] Memory allocated.
[TEST] Block A allocation SUCCESS

[TEST] Allocating Block B: 200 bytes
[MEM] Free block split.
[MEM] Memory allocated.
[TEST] Block B allocation SUCCESS

[MEM] Memory scan completed.
[MEM] Used blocks present.
[MEM] Free blocks available.

[TEST] Freeing Block A
[MEM] Memory released.
[MEM] Memory scan completed.
[MEM] Used blocks present.
[MEM] Free blocks available.

[TEST] Freeing Block B
[MEM] Memory released.
[MEM] Memory scan completed.
[MEM] No used blocks.
[MEM] Free blocks available.

[SYSTEM] Block splitting test completed.

```

Figure 3. Development evidence from the block-splitting and free-block test.

When `kfree()` releases a block, the allocator scans adjacent free regions and coalesces them. Coalescing increases the size of contiguous free memory and reduces external fragmentation.

Before coalescing:
 [FREE] [USED] [FREE]

After the middle block is released:
 [ONE LARGER FREE BLOCK]

4.3 External Fragmentation Measurement

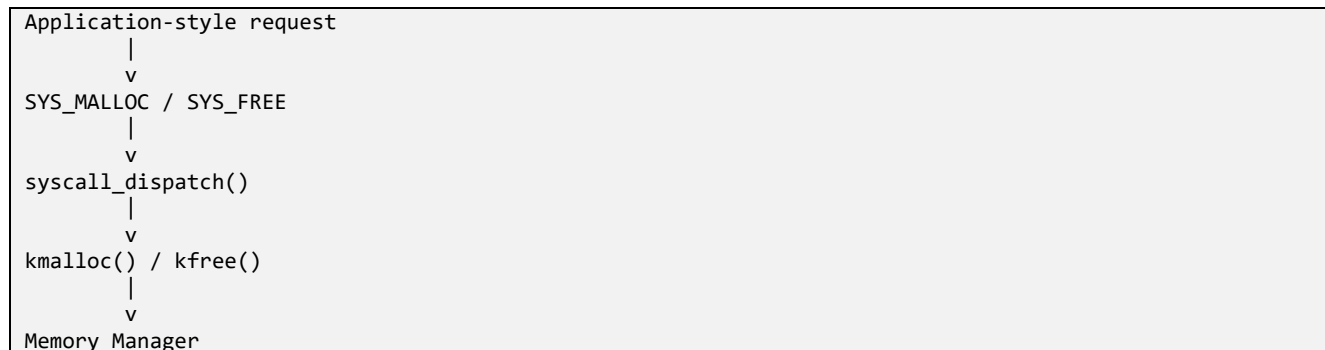
EdgeOS measures external fragmentation using total free memory and the largest individual free block. The integer percentage used by the prototype is:

$$\text{Fragmentation} = 100 - ((\text{Largest Free Block} * 100) / \text{Total Free Memory})$$

In the controlled test, three blocks were allocated: A = 200 KB, B = 300 KB and C = 100 KB. Releasing A and C left B allocated between free regions, producing 28% external fragmentation. Releasing B allowed the allocator to coalesce all adjacent free regions and fragmentation fell to 0%.

4.4 Kernel-Side System Call Dispatcher

The syscall module provides a clear service-dispatch layer between an application-style request in the demo and the underlying memory manager. SYS_MALLOC passes the requested size to kmalloc(), while SYS_FREE passes the provided pointer to kfree(). Unsupported call numbers are rejected as unknown system calls.



4.5 Process Management and PCB

The process manager maintains a fixed-size PCB table. Each PCB records PID, process name, priority, state, allocated memory size and the pointer returned by the memory allocator. Creating a process requests memory, and terminating a process releases that memory.

PCB field	Purpose
PID	Unique process identifier
Name	Human-readable workload name
Priority	LOW, MEDIUM or HIGH scheduling/resource importance
State	UNUSED, READY, RUNNING, WAITING or TERMINATED
Memory size	Bytes assigned to the process
Memory pointer	Address returned by the kernel allocator

4.6 CPU Scheduling

EdgeOS demonstrates two scheduling policies. Priority Scheduling selects the highest-priority READY process first, producing Camera, Sensor and then Logger. Round Robin iterates READY processes cyclically to simulate equal time-slice opportunities.

Policy	Observed order	Purpose
Priority	Camera -> Sensor -> Logger	Shows importance-based selection
Round Robin	Logger -> Camera -> Sensor, repeated	Shows cyclic fairness among READY processes

4.7 Resource-Aware Reclamation

The Resource Manager converts free-memory information into three pressure levels. The current thresholds are NORMAL above 400 KB free, WARNING between 200 KB and 400 KB free, and CRITICAL below 200 KB free. When the handler is invoked during CRITICAL pressure, it searches for a READY LOW-priority process and terminates the first reclaimable candidate.

Pressure level	Free-memory condition	Action in prototype
NORMAL	More than 400 KB	No reclamation
WARNING	200 KB to 400 KB	Report warning state
CRITICAL	Less than 200 KB	Protect higher-priority workloads and reclaim a LOW-priority process

The final resource experiment allocates 64 KB to Camera, 32 KB to Sensor and 760 KB to Logger. This produces a CRITICAL condition. EdgeOS then reclaims Logger, frees its memory, coalesces the resulting region and returns to NORMAL pressure while Camera and Sensor remain READY.

5. Testing and Evaluation

Testing was performed as an integrated boot-time demonstration. Each subsystem prints observable state before and after the operation so the expected result can be compared with the actual output.

Test ID	Test	Expected result	Observed result	Status
T1	Kernel boot	Kernel reaches EdgeOS demo	Boot and debug console initialized	PASS
T2	100/200/300-byte allocation	All three allocations succeed	600-byte payload allocated across three blocks	PASS
T3	Free + coalescing	Free blocks merge when adjacent	Single free region restored after releases	PASS
T4	Fragmentation test	Fragmentation decreases after coalescing	28% -> 0%	PASS
T5	SYS_MALLOC	Dispatcher allocates requested memory	256 bytes allocated	PASS
T6	SYS_FREE	Dispatcher releases memory	Pool returns to zero used payload	PASS
T7	Invalid syscall	Unsupported number rejected	Unknown system call reported	PASS
T8	Priority scheduler	Higher priority scheduled first	Camera -> Sensor -> Logger	PASS
T9	Round Robin	READY processes receive cyclic turns	Two rounds completed	PASS
T10	Resource pressure	Critical pressure triggers low-priority reclamation	Logger reclaimed; pressure NORMAL	PASS

5.1 Quantitative Results

Measured Results from the Final Integrated Demonstration

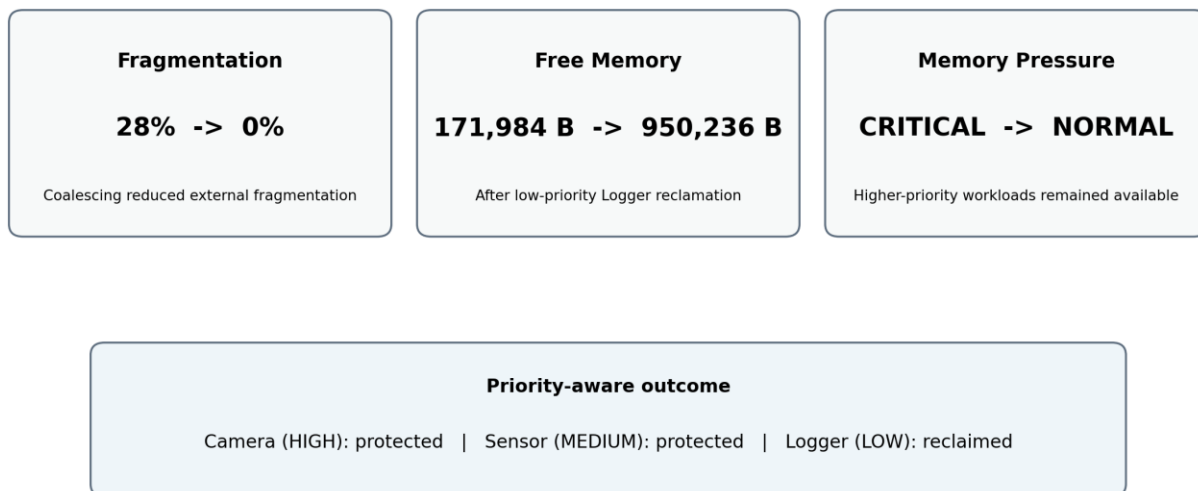


Figure 4. Summary of measured results from the final integrated demonstration.

Metric	Before	After	Interpretation
External fragmentation	28%	0%	Adjacent free regions were coalesced
Free memory under resource test	171,984 bytes	950,236 bytes	Low-priority Logger memory was reclaimed
Memory pressure	CRITICAL	NORMAL	Resource condition recovered
Logger state	READY	TERMINATED	LOW-priority workload reclaimed
Camera state	READY	READY	HIGH-priority workload preserved
Sensor state	READY	READY	MEDIUM-priority workload preserved

5.2 Final Integrated Demo Evidence

The final clean build was verified with make clean and make. The resulting kernel passed the GRUB Multiboot check, a new EdgeOS.iso was created, and make run completed all integrated demonstrations. The final status block was:

```
EDGEOS FINAL DEMO COMPLETE
[OK] Memory Management
[OK] Fragmentation Optimization
[OK] System Call Dispatcher
[OK] Process Management / PCB
[OK] Priority Scheduling
[OK] Round Robin Scheduling
[OK] Resource-Aware Reclamation
```

6. Challenges Faced - STAR Analysis

The submission requires the technical challenges to be described using STAR: Situation, Task, Action and Result. The following examples are directly connected to the development of the final EdgeOS prototype.

Challenge	Situation	Action	Result
QEMU output under WSL	GUI output was unreliable	Moved kernel logging to QEMU debug port 0xE9	Stable terminal-based kernel debugging
Memory waste	Early allocator could consume a whole large free block	Implemented block splitting	Multiple allocations could coexist
External fragmentation	Free memory became separated into regions	Implemented coalescing + fragmentation metric	28% -> 0%
Critical memory pressure	Background workload consumed most of the pool	Added pressure thresholds and LOW-priority reclamation	CRITICAL -> NORMAL; free memory 171,984 -> 950,236 B
Build/reproducibility	Manual compilation and path errors slowed testing	Corrected module structure and created Makefile	Clean build and one-command run workflow

6.1 STAR Challenge 1 - Reliable QEMU Kernel Output

Situation

During early kernel testing in WSL, QEMU could start but its graphical output was unreliable or unresponsive. This made it difficult to distinguish a kernel failure from a display problem.

Task

I needed a dependable way to observe kernel boot messages and subsystem output during every development iteration.

Action

I implemented a debug module that writes characters to QEMU debug port 0xE9 and launched QEMU with -display none and -debugcon stdio. This moved debugging from the graphical window to the WSL terminal.

Result

Kernel output became stable and immediately visible. The same mechanism was then used for memory statistics, process tables, scheduler traces and resource-pressure results throughout the project.

6.2 STAR Challenge 2 - Inefficient Early Memory Allocation

Situation

The initial allocator could locate a free region, but a small request could make an entire large free block unavailable, preventing additional allocations even though unused memory remained.

Task

I needed the memory pool to support several simultaneous allocations while preserving unused space.

Action

I added block splitting. If a free block is larger than the request plus the metadata/minimum remainder, EdgeOS creates a USED block for the request and a new FREE block from the remainder.

Result

The allocator successfully handled 100-byte, 200-byte and 300-byte allocations at the same time. The final demo also reports the number of used/free blocks and remaining memory, making the improvement observable.

6.3 STAR Challenge 3 - External Fragmentation

Situation

Releasing blocks in different positions produced multiple free regions separated by an allocated block. Total free memory was high, but the largest contiguous region was smaller than the total.

Task

I needed to reduce fragmentation and demonstrate the improvement using a measurable metric rather than only visual output.

Action

I implemented adjacent free-block coalescing in the deallocation path and added an external-fragmentation calculation based on the largest free block relative to total free memory.

Result

The controlled test reported 28% fragmentation before the middle block was released. After coalescing all adjacent free blocks, the result became 0%.

6.4 STAR Challenge 4 - Critical Memory Pressure and Workload Importance

Situation

A basic allocator knows whether memory is free but not which workload is important. In the resource test, the LOW-priority Logger consumed 760 KB, leaving only 171,984 bytes free and placing the system in CRITICAL pressure.

Task

I needed the kernel to recover memory while preserving the higher-priority Camera and Sensor workloads.

Action

I added a Resource Manager that classifies pressure from free memory and, during CRITICAL pressure, scans READY processes for a LOW-priority candidate. It terminates Logger through the process manager, which returns Logger memory through kfree() and allows coalescing.

Result

Logger changed from READY to TERMINATED. Free memory increased to 950,236 bytes, memory pressure returned to NORMAL, and Camera and Sensor remained available for priority scheduling.

6.5 STAR Challenge 5 - Build Failure and Reproducibility

Situation

During modularization, a manual build attempt failed when the compiler could not find memory/memory.c. As more modules were added, manually repeating NASM, GCC, linking, GRUB validation, ISO creation and QEMU commands also increased the chance of stale or missing build steps.

```
fahim@FAHIM:~$ cd ~/EdgeOS
fahim@FAHIM:~/EdgeOS$ nasm -f elf32 boot/boot.asm -o boot.o
fahim@FAHIM:~/EdgeOS$ gcc -m32 -ffreestanding -fno-pie -fno-stack-protector \
-c kernel/kernel.c -o kernel.o
fahim@FAHIM:~/EdgeOS$ gcc -m32 -ffreestanding -fno-pie -fno-stack-protector \
-c kernel/debug.c -o debug.o
fahim@FAHIM:~/EdgeOS$ gcc -m32 -ffreestanding -fno-pie -fno-stack-protector \
-c memory/memory.c -o memory.o
cc1: fatal error: memory/memory.c: No such file or directory
compilation terminated.
fahim@FAHIM:~/EdgeOS$ |
```

Figure 5. Development evidence of an early module-path compilation failure.

Task

I needed to correct the project structure and make the complete build reproducible from source for both development and faculty evaluation.

Action

I corrected the source/module layout and created a GNU Makefile containing explicit rules for all object files, kernel linking, Multiboot validation, ISO generation, cleaning and QEMU execution.

Result

The project now rebuilds cleanly with make clean and make, then runs with make run. A fresh EdgeOS.iso was successfully generated from source, proving that the final submission does not depend on stale object files.

7. Technical Scope and Limitations

EdgeOS is an educational prototype rather than a production operating system. Clear scope boundaries are important because they prevent the demonstration from overstating what has been implemented.

Implemented in EdgeOS	Not implemented in this version
Bootable 32-bit x86 kernel	Virtual memory and paging
First-Fit kernel allocator	Memory compaction
Block splitting and coalescing	Ring-3 user-space processes
Fragmentation measurement	Interrupt/trap-based user-to-kernel syscall transition
Kernel-side SYS_MALLOC / SYS_FREE dispatcher	Hardware timer-driven context switching
PCB process table and process memory	Multicore scheduling
Priority and Round Robin scheduling decisions	Persistent filesystem services
Memory-pressure monitoring and low-priority reclamation	Production device drivers and networking stack

8. Skills Demonstrated

Skill area	Evidence in project
Operating-system design	Modular kernel architecture integrating memory, process, scheduling and resource-management subsystems
Low-level programming	Freestanding C and x86 Assembly boot code
Memory management	First Fit, metadata, splitting, free, coalescing and fragmentation measurement
Process management	PCB table, PID, state, priority and process-owned memory
Scheduling	Priority ordering and Round Robin simulation

Skill area	Evidence in project
Kernel interfaces	SYS_MALLOC and SYS_FREE dispatcher with invalid-call handling
Resource-aware design	Memory-pressure thresholds and priority-aware reclamation
Debugging	QEMU debug port, terminal traces and iterative subsystem testing
Build engineering	NASM/GCC/ld/GRUB/xorriso/Make workflow
Technical communication	STAR challenge analysis, measurable results and reproducible demo

9. Conclusion

EdgeOS demonstrates a complete educational progression from kernel booting to resource-aware memory management. The project does not treat allocation as an isolated function. Instead, memory state is connected to process metadata, process priority, scheduling and a simple reclamation policy.

The allocator demonstrated splitting, deallocation and adjacent free-block coalescing, with external fragmentation reduced from 28% to 0% in the controlled test. The Resource Manager demonstrated a second form of optimization: when free memory fell to 171,984 bytes and pressure became CRITICAL, the LOW-priority Logger was reclaimed. Free memory rose to 950,236 bytes, pressure returned to NORMAL, and the Camera and Sensor workloads remained available.

The final Makefile also makes the project reproducible from source using a clean build and one run command. These results show practical skills in kernel programming, memory management, process management, scheduling, debugging and technical problem solving while keeping the limitations of the prototype explicit.

Appendix A. Final Build and Run Commands

From the final project directory, I can rebuild and run EdgeOS using:

```
make clean
make
make run
```